

```

1  '''prmtriangles_solution.py
2
3      This is the PRM solution code for the 2D triangular problem.
4
5      Use the PRM algorithm to find a path around polygonal obstacles.
6
7  '''
8
9  import bisect
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import random
13 import time
14
15 from math          import inf, pi, sin, cos, sqrt, ceil, dist
16
17 from shapely.geometry import Point, LineString, Polygon, MultiPolygon
18 from shapely.prepared import prep
19
20
21 #####
22 #
23 #   Parameters
24 #
25 #   Define the N/K...
26 #
27 #           (b)      (c)      (d)
28 #   N = 50-80  25-40  400
29 #   K =   4      8      8
30 #
31 N = 400
32 K = 2
33 connect_count = 0
34
35
36 #####
37 #
38 #   World Definitions
39 #
40 #   List of obstacles/objects as well as the start/goal.
41 #
42 (xmin, xmax) = (0, 10)
43 (ymin, ymax) = (0, 12)
44

```

```

45 # Collect all the triangles and prepare (for faster checking).
46 obstacles = prep(MultiPolygon([
47     Polygon([[7, 3], [3, 3], [3, 4], [7, 3]]),
48     Polygon([[5, 5], [7, 7], [4, 6], [5, 5]]),
49     Polygon([[9, 2], [8, 7], [6, 5], [9, 2]]),
50     Polygon([[1, 10], [7, 10], [4, 8], [1, 10]])))
51
52 # Define the start/goal states (x, y, theta)
53 (xstart, ystart) = (6, 1)
54 (xgoal, ygoal) = (5, 11)
55
56
57 #####
58 #
59 # Visualization Class
60 #
61 # This renders the world. In particular it provides the methods:
62 # show(text = '') Show the current figure
63 # drawNode(node, **kwargs) Draw a single node
64 # drawEdge(node1, node2, **kwargs) Draw an edge between nodes
65 # drawPath(path, **kwargs) Draw a path (list of nodes)
66 #
67 class Visualization:
68     def __init__(self):
69         # Clear the current, or create a new figure.
70         plt.clf()
71
72         # Create a new axes, enable the grid, and set axis limits.
73         plt.axes()
74         plt.grid(True)
75         plt.gca().axis('on')
76         plt.gca().set_xlim(xmin, xmax)
77         plt.gca().set_ylim(ymin, ymax)
78         plt.gca().set_aspect('equal')
79
80         # Show the triangles.
81         for poly in obstacles.context.geoms:
82             plt.plot(*poly.exterior.xy, 'k-', linewidth=2)
83
84         # Show immediately.
85         self.show()
86
87     def show(self, text = ''):
88         # Show the plot.
89         plt.pause(0.001)
90         # If text is specified, print and wait for confirmation.

```

```

91         if len(text)>0:
92             input(text + ' (hit return to continue)')
93
94     def drawNode(self, node, **kwargs):
95         plt.plot(node.x, node.y, **kwargs)
96
97     def drawEdge(self, head, tail, **kwargs):
98         plt.plot([head.x, tail.x], [head.y, tail.y], **kwargs)
99
100    def drawPath(self, path, **kwargs):
101        for i in range(len(path)-1):
102            self.drawEdge(path[i], path[i+1], **kwargs)
103
104
105    #####
106    #
107    #   A* Planning Algorithm
108    #
109    def astar(nodes, start, goal):
110        # Clear the A* search tree information.
111        for node in nodes:
112            node.done      = False
113            node.seen      = False
114            node.parent    = None
115            node.creach    = 0
116            node.ctogoest  = inf
117
118        # Prepare the still empty *sorted* on-deck queue.
119        onDeck = []
120
121        # Begin with the start node on-deck.
122        start.done      = False
123        start.seen      = True
124        start.parent    = None
125        start.creach    = 0
126        start.ctogoest  = start.costToGoEst(goal)
127        bisect.insort(onDeck, start)
128
129        # Continually expand/build the search tree.
130        while True:
131            # Make sure we have something pending in the on-deck queue.
132            # Otherwise we were unable to find a path!
133            if not (len(onDeck) > 0):
134                return None
135
136            # Grab the next node (first on deck).

```

```

137     node = onDeck.pop(0)
138
139     # Mark this node as done and check if the goal is thereby done.
140     node.done = True
141     if goal.done:
142         break
143
144     # Add the neighbors to the on-deck queue (or update)
145     for neighbor in node.neighbors:
146         # Skip if already done.
147         if neighbor.done:
148             continue
149
150         # Compute the cost to reach the neighbor via this new path.
151         creach = node.creach + node.costToConnect(neighbor)
152
153         # Just add to on-deck if not yet seen (in correct order).
154         if not neighbor.seen:
155             neighbor.seen = True
156             neighbor.parent = node
157             neighbor.creach = creach
158             neighbor.ctogoest = neighbor.costToGoEst(goal)
159             bisect.insort(onDeck, neighbor)
160             continue
161
162         # Skip if the previous path to reach (cost) was same or better!
163         if neighbor.creach <= creach:
164             continue
165
166         # Update the neighbor's connection and resort the on-deck queue.
167         # Note the cost-to-go estimate does not change.
168         neighbor.parent = node
169         neighbor.creach = creach
170         onDeck.remove(neighbor)
171         bisect.insort(onDeck, neighbor)
172
173     # Build the path.
174     path = [goal]
175     while path[0].parent is not None:
176         path.insert(0, path[0].parent)
177
178     # Return the path.
179     return path
180
181
182 #####

```

```

183 #
184 #   Node Definition
185 #
186 class Node():
187     #####
188     # Initialization:
189     def __init__(self, x, y):
190         # Define/remember the state/coordinates (x,y).
191         self.x = x
192         self.y = y
193
194         # Edges = set of neighbors. This needs to be filled in.
195         self.neighbors = set()
196
197         # Clear the status, connection, and costs for the A* search tree.
198         #   TRUNK: done = True
199         #   LEAF:  done = False, seen = True
200         #   AIR:   done = False, seen = False
201         self.done      = False
202         self.seen      = False
203         self.parent    = None
204         self.creach    = 0           # Known/actual cost to get here
205         self.ctogoest  = inf        # Estimated cost to go from here
206
207     #####
208     # A* functions:
209     # Actual cost to connect to a neighbor and estimated to-go cost to
210     # a distant (goal) node.
211     def costToConnect(self, other):
212         return self.distance(other)
213
214     def costToGoEst(self, goal):
215         return self.distance(goal)
216
217     # Define the "less-than" to enable sorting in A*. Use total cost estimate.
218     def __lt__(self, other):
219         return (self.creach + self.ctogoest) < (other.creach + other.ctogoest)
220
221     #####
222     # PRM functions:
223     # Compute the relative distance to another node.
224     def distance(self, other):
225         return sqrt((self.x - other.x)**2 + (self.y - other.y)**2)
226
227     # Check whether in free space.
228     def inFreespace(self):

```

```

229         point = Point(self.x, self.y)
230         return obstacles.disjoint(point)
231
232     # Check the local planner - whether this connects to another node.
233     def connectsTo(self, other):
234         line = LineString([(self.x, self.y), (other.x, other.y)])
235         return obstacles.disjoint(line)
236
237     #####
238     # Utilities:
239     # In case we want to print the node.
240     def __repr__(self):
241         return "<Point %5.2f,%5.2f>" % (self.x, self.y)
242
243     # Compute/create an intermediate node. This can be useful if you
244     # need to check the local planner by testing intermediate nodes.
245     def intermediate(self, other, alpha):
246         return Node(self.x + alpha * (other.x - self.x),
247                     self.y + alpha * (other.y - self.y))
248
249
250     #####
251     #
252     # PRM Functions
253     #
254     # Create and return the list of nodes.
255     def createNodes(N):
256         # Add nodes sampled uniformly across the space.
257         nodes = []
258         while len(nodes) < N:
259             node = Node(random.uniform(xmin, xmax),
260                         random.uniform(ymin, ymax))
261             if node.inFreespace():
262                 nodes.append(node)
263         return nodes
264
265     # Connect to up to K nearest neighbors (classic/standard approach)
266     def connectNearestNeighbors(nodes, K):
267         # Set global variable to count number of connectsTo() calls
268         global connect_count
269
270         # Clear any existing neighbors, which are stored as a set.
271         for node in nodes:
272             node.neighbors = set()
273
274         # Check the neighbors.

```

```

275     for node in nodes:
276         # Examine the K nearest neighbors (ignoring node itself).
277         indicies = np.argsort(np.array([node.distance(n) for n in nodes]))
278
279         # Define total connections for each node and current neighbor
280         total_connect = 0
281         neighbor_index = 1
282
283         # Check if total connections has reached K neighbors for given node
284         while total_connect < K:
285             neighbor = nodes[indicies[neighbor_index]]
286             # Force a undirected graph, so node is also neighbor's neighbor.
287             if neighbor not in node.neighbors:
288                 connect_count += 1
289                 if node.connectsTo(neighbor):
290                     node.neighbors.add(neighbor)
291                     neighbor.neighbors.add(node)
292                     total_connect += 1          # If connection successful,
293                                             # increase number of connections by 1
294                 neighbor_index += 1          # Move to next neighbor in list
295
296     # Compute the path cost
297     def pathCost(path):
298         cost = 0
299         for i in range(1, len(path)):
300             cost += path[i-1].costToConnect(path[i])
301         return cost
302
303     # Post process the path
304     def postProcess(path):
305         # FIXME: This is just a placeholder. You should replace with
306         # actual processing.
307
308         i = 1
309         while i < (len(path) - 1):          # Don't include the start and goal nodes
310             predecessor = path[i - 1]
311             successor = path[i + 1]
312             if (predecessor.connectsTo(successor)):
313                 path.pop(i)          # Pop the node, but don't increment i
314                                     # since next node will fall into its place
315             else:
316                 i += 1
317
318     finalpath = path
319     return finalpath
320

```

```

321 #####
322 #
323 #
324 # Main Code
325 #
326 def main():
327     # Report the parameters.
328
329     print('Running with', N, 'nodes and', K, 'neighbors.')
330
331     # Create the figure. Some computers seem to need an additional show()?
332     visual = Visualization()
333     visual.show()
334
335     # Create the start/goal nodes.
336     startnode = Node(xstart, ystart)
337     goalnode = Node(xgoal, ygoal)
338
339     # Show the start/goal nodes.
340     visual.drawNode(startnode, color='orange', marker='o')
341     visual.drawNode(goalnode, color='purple', marker='o')
342     visual.show("Showing basic world")
343
344
345     # Create the list of nodes.
346     print("Sampling the nodes...")
347     tic = time.time()
348     nodes = createNodes(N)
349     toc = time.time()
350     print("Sampled the nodes in %fsec." % (toc-tic))
351
352     # Show the sample nodes.
353     for node in nodes:
354         visual.drawNode(node, color='k', marker='x')
355     visual.show("Showing the nodes")
356
357     # Add the start/goal nodes.
358     nodes.append(startnode)
359     nodes.append(goalnode)
360
361
362     # Connect to the nearest neighbors.
363     print("Connecting the nodes...")
364     tic = time.time()
365     connectNearestNeighbors(nodes, K)
366     toc = time.time()

```

```

367     print("Connected the nodes in %fsec." % (toc-tic))
368
369     # Show the neighbor connections.
370     for (i,node) in enumerate(nodes):
371         for neighbor in node.neighbors:
372             if neighbor not in nodes[:i]:
373                 visual.drawEdge(node, neighbor, color='g', linewidth=0.5)
374     visual.show("Showing the full graph")
375
376
377     # Run the A* planner.
378     print("Running A*...")
379     tic = time.time()
380     path = astar(nodes, startnode, goalnode)
381     toc = time.time()
382     print("Ran A* in %fsec." % (toc-tic))
383
384     # If unable to connect, show the part explored.
385     if not path:
386         print("UNABLE TO FIND A PATH")
387         print(f"Number of connectsTo() calls: {connect_count}")
388         for node in nodes:
389             if node.done:
390                 visual.drawNode(node, color='r', marker='o')
391         visual.show("Showing DONE nodes")
392         return
393
394     # Show the path.
395     cost = pathCost(path)
396     visual.drawPath(path, color='r', linewidth=2)
397     visual.show("Showing the raw path (cost/length %.1f)" % cost)
398
399
400     # Post process the path.
401     finalpath = postProcess(path)
402
403     # Show the post-processed path.
404     cost = pathCost(finalpath)
405     visual.drawPath(finalpath, color='b', linewidth=2)
406     visual.show("Showing the post-processed path (cost/length %.1f)" % cost)
407     print(f"Number of connectsTo() calls: {connect_count}")
408
409 if __name__ == "__main__":
410     main()

```